BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Bùi Quyền Kiệt
MSSV / Lớp	3124720036

Mục tiêu học tập

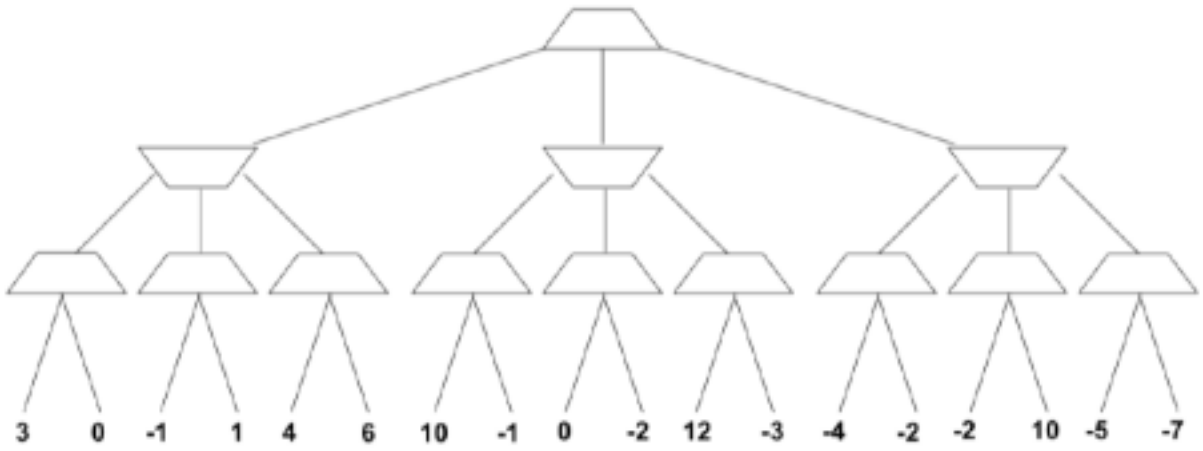
• Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa. • Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.

• Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP. • Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi. • Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

- Cây gồm 3 tầng quyết định: Tầng gốc (MAX) -> Tầng trung gian 1 (MIN: L, M, R) -> Tầng trung gian 2 (MAX: 10 nút con) -> Các nút lá.

Tầng MAX 2 (Trái): Nút 1 = max(3) = 3; Nút 2 = max(0, -1) = 0; Nút 3 = max(1, 4) = 4.

Tầng MAX 2 (Giữa): Nút 4 = max(6, 10) = 10; Nút 5 = max(-1, 0) = 0; Nút 6 = max(-2, 12) = 12.

Tầng MAX 2 (Phải): Nút 7 = max(-3, -4) = -3; Nút 8 = max(-2, -2) = -2; Nút 9 = max(10, -5) = 10; Nút 10 = max(-7) = -7.

Tầng MIN 1 (L): Nhánh Trái L = min(3, 0, 4) = 0

Tầng MIN 1 (M): Nhánh Giữa M = min(10, 0, 12) = 0

Tầng MIN 1 (R): Nhánh Phải R = min(-3, -2, 10, -7) = -7

Nút Gốc (MAX): Gốc (MAX) = max(L, M, R) = max(0, 0, -7) = 0.

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

- Lựa chọn tối ưu: MAX có thể chọn nhánh Trái (L) hoặc nhánh Giữa (M) vì cả hai đều đạt giá trị tối ưu bằng 0.

Giá trị trò chơi (Game Value): 0.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

- Khởi tạo tại gốc: $\alpha = -\infty$, $\beta = +\infty$. Quy tắc cắt tỉa: Cắt nhánh khi $\alpha \geq \beta$.

Tại Nhánh Trái (L): Duyệt toàn bộ 3 nút con MAX. Giá trị nút L = 0. Cập nhật tại Gốc: $\alpha = \max(-\infty, 0) = 0$, $\beta = +\infty$.

Tại Nhánh Giữa (M): Duyệt nút con 1 (giá trị 10) và nút con 2 (giá trị 0). Nút MIN (M) nhận giá trị tạm thời ≤ 0 (do min với 0). Lúc này tại M có $\beta_M \leq 0$. So sánh với gốc có $\alpha = 0 \Rightarrow \alpha \geq \beta$ $(0 \geq 0) \Rightarrow$ Cắt tỉa nhánh con thứ 3 của M (nút chứa lá -3, 12). Cắt tỉa nhánh chứa lá -3.

Tại Nhánh Phải (R): Duyệt nút con 1 (giá trị -3). Nút MIN (R) nhận giá trị tạm thời ≤ -3. Lúc này tại R có $\beta_R \leq -3$. So sánh với gốc có $\alpha = 0 \Rightarrow \alpha \geq \beta$ $(0 \geq -3) \Rightarrow$ Cắt tỉa ngay lập tức tất cả các nhánh con còn lại của R (gồm các nút chứa lá -4, -2, -2, 10, -5, -7).

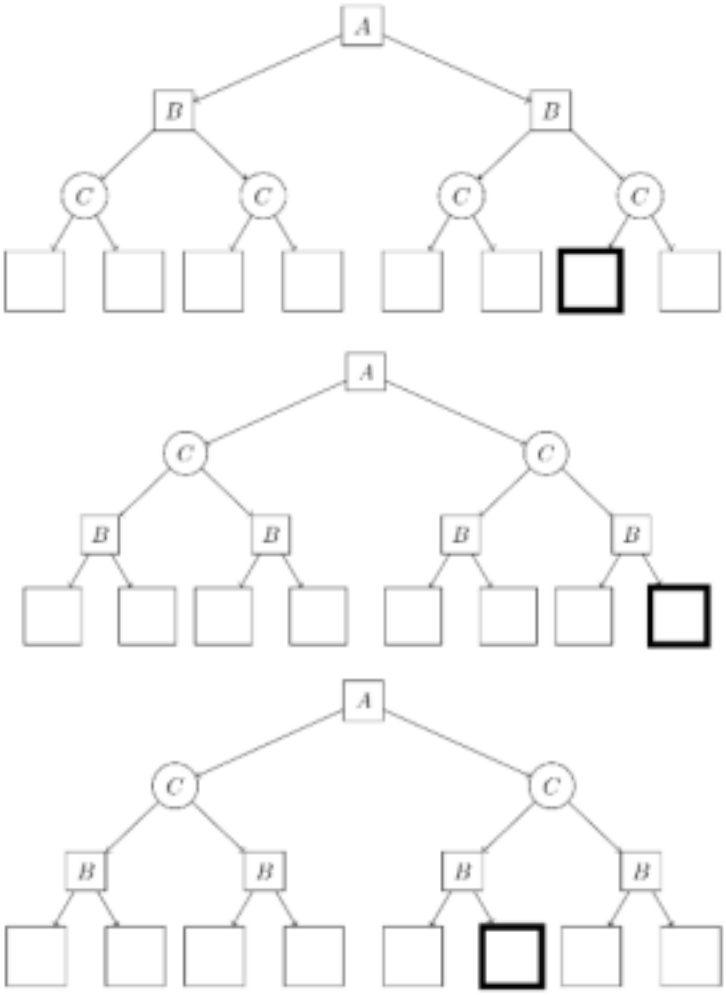
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

- Các nút lá bị cắt tỉa: -3 (lá cuối của M), -4, -2, -2, 10, -5, -7 (toàn bộ các lá còn lại của R). Tổng cộng 7 nút lá bị cắt tỉa.

So sánh: Minimax đánh giá tất cả 17 nút lá. Alpha-Beta Pruning chỉ cần đánh giá 10 nút lá (tiết kiệm được ~41% số lượt đánh giá).

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi “Không thể” và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Kết quả: Không thể.

- Giải thích: Cây có cấu trúc A (MIN) -> B (MAX) -> C (Chance). Nút C tính giá trị kỳ vọng (trung bình cộng của các lá con). Để cắt tỉa một lá thuộc nút Chance C, ta cần biết giá trị giới hạn dưới/trên của C. Tuy nhiên, vì giá trị nút lá chưa thăm có thể là bất kỳ số hữu hạn nào ($-\infty$ đến $+\infty$), một lá chưa thăm có thể kéo giá trị trung bình của C lên cực kỳ cao (hoặc cực kỳ thấp), làm thay đổi hoàn toàn giá trị của nút MAX B phía trên. Do đó, không thể chắc chắn cắt tỉa lá trong nút Chance mà không có giới hạn (bound) trên tiện ích của các lá.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Kết quả: Có thể gán giá trị.

- Ví dụ giá trị lá & Điều kiện: Gán các lá từ trái sang phải như sau: C1 có 2 con B1 (lá 10, 10 -> B1=10) và B2 (lá 10, 10 -> B2=10) $\Rightarrow$ C1 = (10+10)/2 = 10. Giá trị tại A (MIN) nhận tạm ≤ 10. Ở nhánh phải C2: B3 có các lá (0, 0) $\Rightarrow$ B3 = 0. B4 là nút MAX, lá đầu tiên của B4 có giá trị 10. Do B4 là nút MAX nên B4 = max(10, lá_2) ≥ 10. Vì B4 ≥ 10 và B3 = 0, giá trị của C2 = (0 + B4)/2 ≥ 5. Vì C2 chưa chắc vượt 10 nên để cắt tỉa lá 2 của B4, ta cần B4 làm cho C2 $\geq 10 \Rightarrow$ B4 ≥ 20. Nếu lá 1 của B4 = 20, thì B4 $\geq 20 \Rightarrow$ C2 $\geq$ (0+20)/2 = 10. Khi đó C2 ≥ 10, mà A (MIN) đã có nhánh trái = 10, nên A sẽ chọn min(10, C2) = 10. Giá trị B4 không thể làm A chọn giá trị tốt hơn 10 cho MIN. Do đó, nếu gán lá 1 của B4 = 20 (hoặc lớn hơn), lá 2 của B4 sẽ bị cắt tỉa.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

- Kết quả: Có thể gán giá trị.

- Ví dụ giá trị lá & Điều kiện: Gán nhánh trái C1 = 10 (A nhận tạm ≤ 10). Ở C2, nút B3 (MAX) có lá thứ nhất = 20. Vì B3 là MAX nên B3 = max(20, lá_2) ≥ 20. Do B3 ≥ 20, giá trị kỳ vọng C2 = (B3 + B4)/2 ≥ (20 + B4)/2. Nếu B4 ≥ 0 (ví dụ các lá B4 là 0), thì C2 ≥ 10. Khi C2 ≥ 10, nút A (MIN) có A = min(10, C2) = 10. Bất kể lá thứ 2 của B3 có giá trị lớn đến đâu (ví dụ 100), B3 vẫn ≥ 20 và C2 vẫn ≥ 10, không làm thay đổi quyết định của A. Do đó lá thứ 2 của B3 bị cắt tỉa.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

- Trong cây Minimax thông thường, các nút MIN/MAX lấy giá trị chằn chặn (bounding / monotonic limits) nên chỉ cần một nhánh con đạt ngưỡng α hoặc β là có thể cắt tỉa ngay lập tức. Ngược lại, trong Expectimax, nút Chance lấy trung bình cộng có trọng số của tất cả các nhánh con. Một giá trị lá chưa thăm lớn/nhỏ tùy ý có thể làm thay đổi trung bình cộng. Cắt tỉa chỉ xảy ra khi ta có giới hạn trên/dưới cố định cho tiện ích lá (bounded utilities) hoặc khi giá trị hiện tại của các nhánh đã thăm đủ lớn/nhỏ để đảm bảo giá trị kỳ vọng chắc chắn vượt ngưỡng cắt tỉa.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $\square\square_{\square\square} \leq \square\square_{\square\square}$. ☐ Đúng ■ Sai

- Giải thích: Minimax giả định đối thủ MIN chọn hành động xấu nhất cho MAX (tối thiểu hóa tiện ích). Expectimax giả định đối thủ (hoặc môi trường) hành động theo một phân phối xác suất ngẫu nhiên. Trung bình cộng theo xác suất luôn lớn hơn hoặc bằng giá trị nhỏ nhất (phần tử tối thiểu) của một tập hợp. Do đó, đối với cùng một cây, giá trị Expectimax V_E luôn lớn hơn hoặc bằng giá trị Minimax V_M (V_M ≤ V_E). Khẳng định V_M ≤ V_E là ĐÚNG. (Chọn ĐÚNG).

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là $\square\square_{\square\square}$. ■ Đúng ☐ Sai

Giải thích: Chiến lược Minimax được thiết kế cho trường hợp xấu nhất (worst-case guarantee). Nếu môi trường/đối thủ không chơi tối ưu chống lại ta mà hành động ngẫu nhiên, lợi ích nhận được chỉ có thể bằng hoặc cao hơn so với khi đối thủ chơi tối ưu nhất. Do đó, lợi ích kỳ vọng thu được chắc chắn ≥ V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ■ Sai

V_E là lợi ích kỳ vọng đạt được khi dùng chính sách Expectimax tối ưu (chính sách thiết kế riêng cho phân phối xác suất đó). Chính sách Minimax tập trung vào an toàn trường hợp xấu nhất nên có thể bỏ qua các cơ hội thu lợi lớn ngẫu nhiên, do đó lợi ích kỳ vọng của chính sách Minimax có thể nhỏ hơn V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

■ Expectimax với tiện ích trạng thái cuối +1/−1

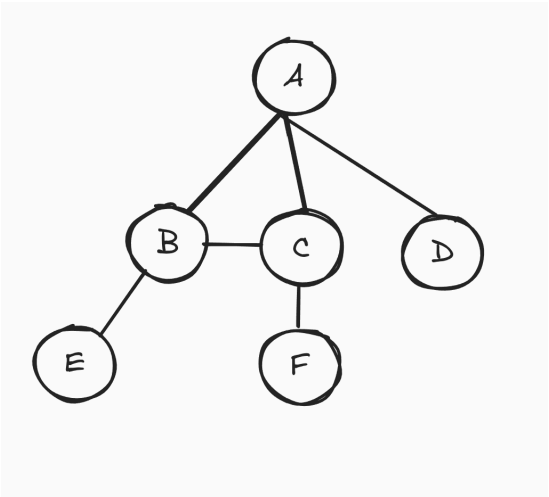
■ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua ☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.
Trò chơi một người với kết quả ngẫu nhiên/không xác định chính là một quá trình quyết định Markov (MDP). Expectimax phù hợp cho các bài toán dạng cây quyết định hữu hạn bước. Value Iteration là thuật toán quy hoạch động tổng quát cho MDP (kể cả có chu trình hoặc vô hạn bước) để tìm chiến lược tối đa hóa lợi ích kỳ vọng tích lũy.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Đáp án: {B, C, D}.

Giải thích: Forward Checking chỉ kiểm tra và lọc miền giá trị của các biến lân cận trực tiếp (có cạnh nối trực tiếp với A) trong đồ thị ràng buộc. Các đỉnh kề với A là B, C, D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?
Đáp án: {C, E}.
Giải thích: Các biến kề trực tiếp với B là A, C, E. Do A đã được gán giá trị cố định từ trước, FC tại B chỉ làm thu hẹp miền giá trị của các biến chưa gán kề với B, tức là C và E.

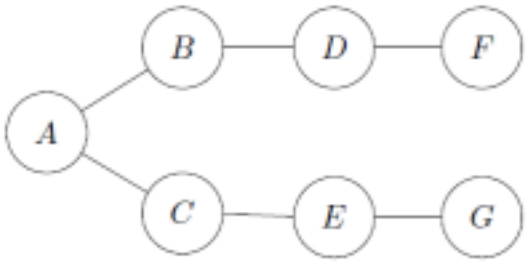
d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Đáp án: {B, C, D, E, F} (Tất cả các biến chưa gán còn lại).

Giải thích: Sau khi gán A, FC thu hẹp miền của B, C, D. Việc thu hẹp B và C sẽ kích hoạt lan truyền ràng buộc (constraint propagation) đến các biến kề với B và C là E và F (thông qua các cung B->E, C->F, B->C). Do đó, miền của E và F cũng có thể bị thu hẹp.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?
Đáp án: {C, D, E, F} (Tất cả các biến chưa gán còn lại).
Giải thích: Gán giá trị cho B sẽ làm thay đổi miền của B thành 1 giá trị duy nhất. Việc này lan truyền Arc Consistency đến các biến hàng xóm C, E, và tiếp tục lan truyền từ C sang F, D.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

f) Thứ tự A → B → C → D → E → F → G

Số lần Backtrack: 0 lần backtrack.

Giải thích: Ban đầu chạy AC trên đồ thị dạng cây làm cho toàn bộ mạng Arc Consistent. Khi gán 1st (A), 2nd (B) -> chạy AC -> lan truyền trên cây làm cho các biến còn lại hoàn toàn nhất quán, không xảy ra xung đột ở bất kỳ bước nào. Do đó 0 backtrack.

g) Thứ tự G → E → C → A → B → D → F

Số lần Backtrack: 0 lần backtrack.

Giải thích: Đây là thứ tự ngược từ lá lên gốc (hoặc từ một nhánh sang gốc). Vì AC ban đầu đã làm sạch tất cả các giá trị không tương thích trên đồ thị cây, và AC được chạy lại sau mỗi 2 biến gán, không có gán sai lầm nào dẫn đến bế tắc.

h) Thứ tự G → F → E → D → C → B → A

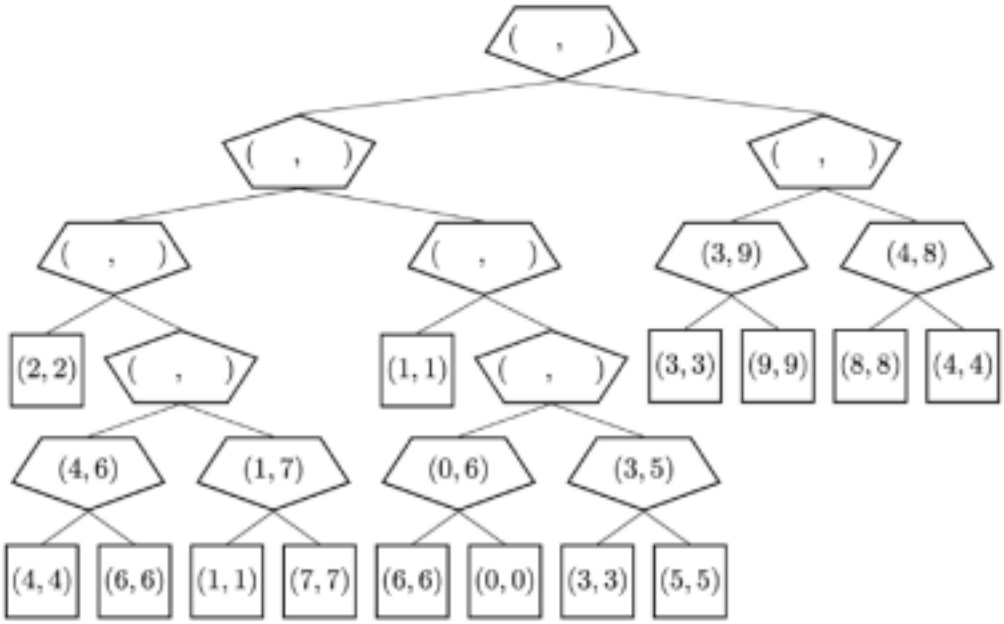
Số lần Backtrack trường hợp xấu nhất: O(d^2) hoặc O(d) tùy thuộc vào độ chặt ràng buộc, tối đa d^2 lần backtrack ở các bước lẻ giữa các lần duy trì AC.

Giải thích: Việc gán luân phiên giữa 2 nhánh độc lập (G ở nhánh 2, F ở nhánh 1, E ở nhánh 2, D ở nhánh 1) khiến các bước gán lẻ (như gán G xong gán F) không được kích hoạt AC ngay lập tức. Do AC chỉ chạy sau các bước

chẵn (2, 4, 6), xung đột ở bước lẻ giữa các nhánh chưa được phát hiện ngay, dẫn đến phải backtrack trong phạm vi miền giá trị d của từng cặp biến.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Quy tắc: Nút MAX (Pacman) chọn nhánh có u lớn nhất. Nút Casper (thực tế muốn giúp Pacman) chọn nhánh có v lớn nhất, nhưng Pacman TIN RẰNG Casper chọn nhánh có u nhỏ nhất.

Tầng 4 (Casper): Nút Casper dưới trái 1 (các lá (4,4), (6,6)): Pacman tin Casper chọn min u => u = min(4, 6) = 4. Casper thực tế chọn max v => v = max(4, 6) = 6. Vậy nút này có cặp (u, v) = (4, 6).

Tầng 4 (Casper): Nút Casper dưới trái 2 (các lá (1,1), (7,7)): u = min(1, 7) = 1, v = max(1, 7) = 7. Cặp (u, v) = (1, 7).

Tầng 4 (Casper): Nút Casper dưới giữa 1 (các lá (6,6), (0,0)): u = min(6, 0) = 0, v = max(6, 0) = 6. Cặp (u, v) = (0, 6).

Tầng 4 (Casper): Nút Casper dưới giữa 2 (các lá (3,3), (5,5)): u = min(3, 5) = 3, v = max(3, 5) = 5. Cặp (u, v) = (3, 5).

Tầng 3 (MAX - Pacman): Nút MAX tầng 3 (Trái): giữa lá (2,2) và nút Casper (4,6) -> Pacman chọn max u => u = max(2, 4) = 4, v = 6. Cặp (4, 6).

Tầng 3 (MAX - Pacman): Nút MAX tầng 3 (Giữa): giữa lá (1,1) và nút Casper (0,6) -> u = max(1, 0) = 1, v = 1.

Cặp (1, 1).

Tầng 2 (Casper): Nút Casper tầng 2 (Trái): chọn giữa nút (4,6) và nút (1,1). Pacman tin Casper chọn min u = min(4, 1) = 1. Casper thực tế chọn max v = max(6, 1) = 6 (chọn nhánh trái). Cặp (1, 6).

Tầng 2 (Casper): Nút Casper tầng 2 (Phải): giữa nút (3,9) và nút (4,8). Pacman tin Casper chọn min u = min(3, 4) = 3. Casper thực tế chọn max v = max(9, 8) = 9 (chọn nhánh trái). Cặp (3, 9).

Nút Gốc (MAX - Pacman): Nút Gốc (MAX - Pacman): so sánh nhánh trái (u=1, v=6) và nhánh phải (u=3, v=9). Pacman chọn max u => chọn nhánh phải (u=3). Do đó giá trị tại Gốc thu được là (u, v) = (3, 9).

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Vì Casper biết chính xác mô hình suy luận của Pacman, Casper có thể cắt tỉa Alpha-Beta khi duyệt cây để dự đoán quyết định của Pacman. Các lá/nhánh bị cắt tỉa bao gồm các nhánh không ảnh hưởng đến giá trị u mà Pacman dùng để ra quyết định.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Cả Pacman và Casper đều làm việc cùng nhau để tối đa hóa tiện ích. Cây trở thành trò chơi hợp tác hoàn toàn (MAX-MAX). Giá trị tại nút gốc sẽ đạt tối đa toàn cục v = 9 (hoặc cao hơn nếu chọn nhánh trái v=6 vs v=7). Pacman sẽ tin tưởng chọn nhánh đem lại tiện ích thực tế cao nhất.

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Đầy đủ nếu hệ số phân nhánh b hữu hạn.	Tối ưu nếu chi phí mỗi bước là đồng nhất (cơ bản bằng c > 0). Không tối ưu nếu chi phí các cạnh khác nhau.
Depth-First Search (DFS)	Đầy đủ trên không gian trạng thái hữu hạn (Tree Search có thể lặp vô hạn nếu có chu trình; Graph Search đầy đủ nếu hữu hạn). Không đầy đủ nếu không gian vô hạn.	Không tối ưu (thường tìm thấy đường đi không ngắn nhất trước).
Iterative Deepening Search (IDS)	Đầy đủ nếu hệ số phân nhánh b hữu hạn.	Tối ưu nếu chi phí mỗi bước đồng nhất (giống BFS nhưng tiết kiệm bộ nhớ).
A* Search	Đầy đủ trên không gian hữu hạn hoặc khi chi phí mỗi bước ε > 0.	Tối ưu nếu heuristic h(n) Admissible (cho Tree Search) hoặc Consistent (cho Graph Search).

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.
Chi phí bước (Step Cost): Nếu chi phí bước c = 0 hoặc có cạnh âm, thuật toán như BFS/IDS không còn đảm bảo tính tối ưu theo chi phí. Ngoài ra, chi phí phải ≥ ε > 0 để tránh đường đi vô hạn có tổng chi phí hữu hạn làm ảnh hưởng tính đầy đủ.
Đặc tính Heuristic: Đặc tính chấp nhận được (Admissibility: h(n) ≤ h*(n)) đảm bảo A* không bỏ qua đường đi tối ưu trong Tree Search. Đặc tính nhất quán (Consistency: h(n) ≤ c(n,a,n') + h(n')) đảm bảo giá trị f(n) tăng đơn điệu trong Graph Search, không cần mở lại nút đã duyệt.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Ảnh hưởng: Chi phí tối ưu có thể tăng hoặc giữ nguyên (nếu có đường đi thay thế tốt hơn).

Cách cập nhật: Cập nhật g(Y) = g(Y) + n. Đưa các nút bị ảnh hưởng bởi X->Y trở lại tập OPEN (hoặc cập nhật lại g-value của chúng) và tiếp tục A* từ tập OPEN hiện tại mà không cần chạy lại từ gốc (dùng thuật toán A* động / D* / LPA*).

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Ảnh hưởng: Đường đi tối ưu KHÔNG thay đổi về cấu trúc nút, nhưng tổng chi phí tối ưu GIẢM đúng n.

Cách cập nhật: Chi phí mới = C_old - n. Không cần chạy lại A*.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Ảnh hưởng: Đường đi tối ưu và chi phí tối ưu HOÀN TOÀN KHÔNG THAY ĐỔI.

Cách cập nhật: Không cần làm gì cả.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Ảnh hưởng: Chi phí tối ưu mới có thể giảm nếu đường đi qua X->Y trở nên tốt hơn đường tối ưu cũ.

Cách cập nhật: Cập nhật g(Y) mới = g(X) + (c - n). Nếu g_new(Y) < g_old(Y), thêm/cập nhật Y vào OPEN và chạy tiếp A* để lan truyền chi phí mới.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Ảnh hưởng: Đường đi tối ưu cũ vẫn tối ưu, chi phí không đổi.

Cách cập nhật: Không cần hành động.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Ảnh hưởng: Đường đi qua X->Y có thể trở thành đường tối ưu mới nếu g(X) + (c - n) + h(Y) < C_old.

Cách cập nhật: Nếu X đã nằm trong OPEN hoặc CLOSED, tính g_potential(Y) = g(X) + (c - n). Nếu giá trị này nhỏ hơn chi phí tối ưu hiện tại, đưa Y vào OPEN với f(Y) = g_potential(Y) + h(Y) và tiếp tục A*.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: Tác tử B và C (và cả A trong môi trường tĩnh không đổi).

Giải thích: Vì môi trường quan sát đầy đủ và tĩnh, percept hiện tại chứa toàn bộ thông tin cần thiết về trạng thái thế giới. Do đó, tác tử phản xạ B (f_B(percept)) là đủ để đưa ra hành động tối ưu hoàn hảo mà không cần nhớ lịch sử.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: Lớp tác tử C.
Giải thích: Khi môi trường bị quan sát một phần, percept hiện tại không đủ để xác định trạng thái thực sự. Tác tử C lưu giữ toàn bộ lịch sử percepts giúp xây dựng/khôi phục trạng thái nội bộ (internal state), từ đó hành động hợp lý.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: Lớp tác tử C.

Giải thích: Yêu cầu lưu trữ lịch sử percept để tính toán phân phối xác suất niềm tin (belief state) về trạng thái hiện tại của môi trường ngẫu nhiên nhằm tối đa hóa tiện ích kỳ vọng.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Khẳng định: ĐÚNG.

Ví dụ & Phản biện: Tồn tại các môi trường 'tầm thường' (trivial environment) nơi mà mọi hành động đều đem lại tiện ích/kết quả như nhau (ví dụ: môi trường chỉ có 1 hành động duy nhất, hoặc mọi hành động đều đạt điểm tối đa +100). Trong môi trường đó, bất kỳ tác tử nào (kể cả tác tử A chọn ngẫu nhiên) cũng đều hành động hợp lý hoàn hảo.

— Hết —